

Sindh Madressatul Islam University Karachi

Object Oriented Programming



SPRING 2026

Lecture 5 of 16
Engr. Majid Kaleem

WEEK 05 - AGENDA

- Static keyword in Java

<https://onecompiler.com/java>

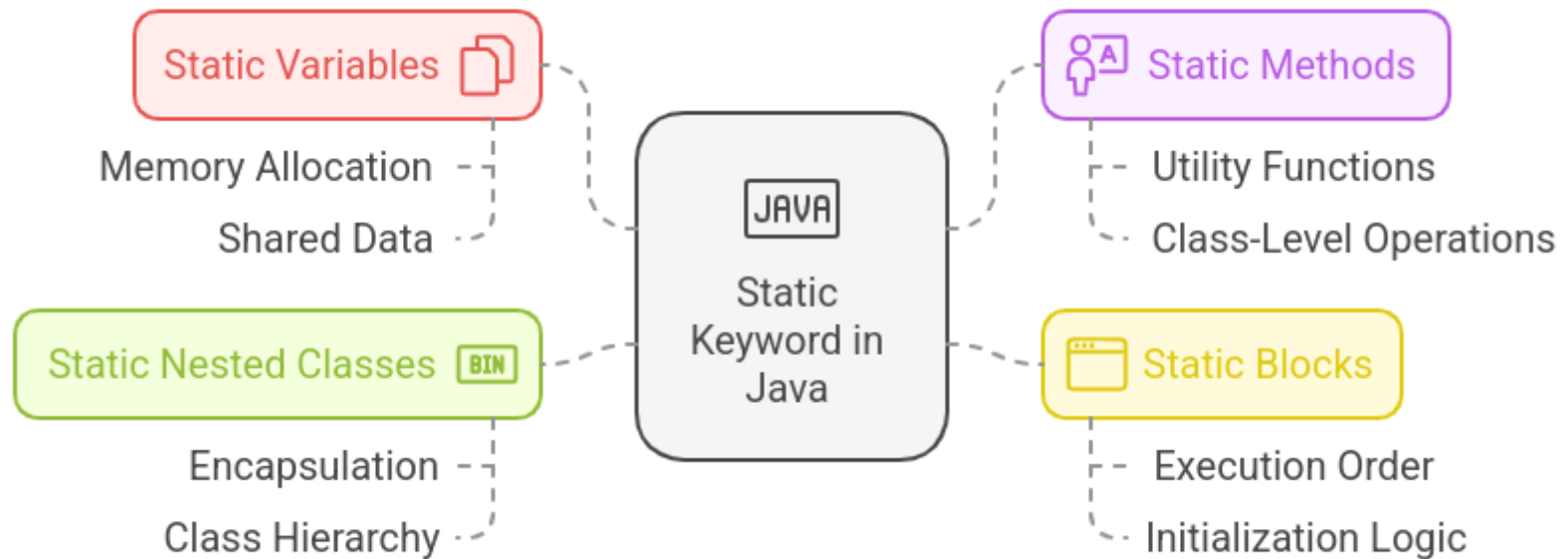
WEEKLY AGENDA

TENTATIVE WEEKLY DATES		TENTATIVE TOPICS
1	26 th -Jan – 30 th Jan	INTRO. TO OBJECT-ORIENTED PROGRAMMING, PROCEDURAL VS.OOP, OOP PRINCIPLES, ETC.
2	02 nd Feb – 06 th Feb	CLASSES & OBJECTS, JAVA NAMING CONVENTION, JAVA PACKAGES, & THE FOUR PILLARS OF OOP
3	09 th Feb – 13 th Feb	METHODS & MODULAR PROGRAMMING
4	16 th Feb – 20 th Feb	ABSTRACT CLASSES & INTERFACES ASSIGNMENT #1 & QUIZ #1
5	23 rd Feb – 27 th Feb	USE OF STATIC KEYWORD IN JAVA
6	02 nd Mar – 06 th Mar	POLYMORPHISM
7	09 th Mar – 13 th Mar	INTERFACES ASSIGNMENT #2 & QUIZ #2
8	16 th Mar – 20 th Mar	EXCEPTION HANDLING
← MID TERM EXAMINATIONS (23rd MAR 26 – 27th MAR 2026) →		
9	30 th Mar – 03 rd Apr	COLLECTIONS FRAMEWORK
10	06 th Apr – 10 th Apr	FILE HANDLING
11	13 th Apr – 17 th Apr	GUI PROGRAMMING
12	20 th Apr – 24 th Apr	APPLICATION DEVELOPMENT ASSIGNMENT #3 & QUIZ #3
13	27 th Apr – 01 st May	DESIGN PRINCIPLES & PATTERNS
14	04 th May – 08 th May	ADVANCED OOP CONCEPTS
15	11 th May – 15 th May	CASE STUDIES AND INDUSTRY PRACTICES ASSIGNMENT #4 & QUIZ #4
16	18 th May – 22 nd May	PRESENTATIONS & REVISION WEEK
← FINAL TERM EXAMINATIONS (25th MAY 26 – 29th MAY 2026) →		

Note

- This schedule is subject to revisions as conditions may warrant.*
- Topics will be covered in sequence no matter if the university observes any planned or unplanned holidays.*
- The information in this course outline is subject to revision as conditions may warrant.*

STATIC KEYWORD IN JAVA



```
import static java.lang.Math.*;           //static import
```

```
public class AllAboutStaticExample {
```

```
    private static int num;                //static variable
```

```
    static {  
        num=4;                             //static block  
        System.out.println("I am a static block");  
    }
```

```
    public static void m1() {                //static method  
        System.out.println("I am a static method");  
    }
```

```
    public static void main(String[] str) {    //main method  
        System.out.println("Using static Import : " + sqrt(num));  
        System.out.println("I am main() method");  
    }
```

```
    static class MyStaticClass {              //static inner class  
        public static void main(String[] str) {  
            System.out.println("I am inside static class");  
        }  
    }
```

```
    interface Java8Interface { //static method inside interface  
        static void m2() {  
            System.out.println("I am inside an interface");  
        }  
    }  
}
```

DON'T FORGET...

Static Element	Why Use It?	When to Use It?
Static Field (Variable)	Shared across all objects; saves memory; represents class-level state.	When you need a common value for all objects (e.g., a counter, configuration flag).
Static Method	Can be called without creating an object; utility/helper functions.	When behavior doesn't depend on object state (e.g., <code>Math.sqrt()</code> , utility methods).
Static Block	Executes once when the class is loaded; used for initialization.	When you need to initialize static fields or run setup code before <i>main()</i> .
Static Nested Class	Groups related classes; can access outer class's static members directly.	When a class is only useful in the context of its outer class, but doesn't need an outer object.
Static Method Overloading	Allows multiple versions of a static method with different parameters.	When you want flexible utility methods with different input signatures.
Static Method Hiding	Subclass defines a static method with the same signature, hiding parent's method.	When you want to redefine behavior at the class level (though rarely recommended).

RULES ABOUT STATIC IN JAVA

- Top-level classes (the ones declared directly in a **.java** file) **cannot be static**.
 - Only nested classes (classes inside another class) can be declared static.
 - Inner classes can be static; these are called static nested classes.
 - Methods can be declared static.
 - Fields (variables) can be declared static.
 - Blocks can be declared static (static initialization blocks).
- Valid combinations are:
 - A static nested class inside another class.
 - Static fields inside a class.
 - Static methods inside a class.
 - Static blocks inside a class.

Syntax:

```
class OuterClass {  
    .....  
    class NestedClass {  
        .....  
    }  
}
```

DON'T FORGET...

- A static variable is a **shared**, class-level variable that **retains** its value across instances of the class.
- Static methods are bound to the **class** and **cannot be overridden**. They can, however, be hidden by a method in a subclass with the same name.
- Static methods are not tied to any instance, so they cannot reference instance-specific data.
- Java does not allow static variables inside methods; they must be declared at the class level.
- The **'this'** keyword is instance-specific and **cannot** be used in a static context.

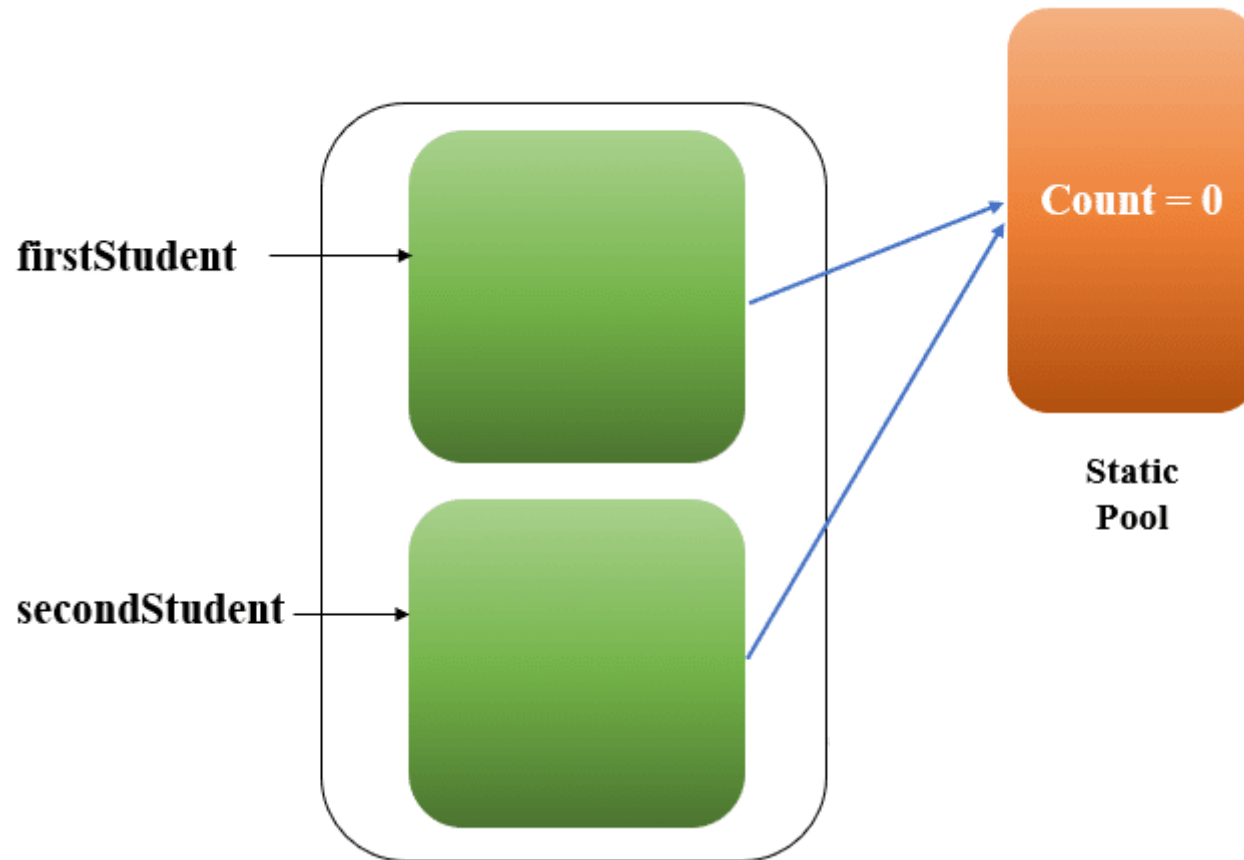
STATIC KEYWORD IN JAVA

- The static keyword is a critical feature in Java, allowing developers to define members (variables, methods, blocks, and nested classes) at the **class** level rather than the **instance (object)** level.
- The static keyword signifies that a member belongs to the class itself, not to individual instances of the class.
- This means:
 1. Static Variables
 2. Static Methods
 3. Static Blocks
 4. Static Nested Classes

STATIC VARIABLES

- Static variables are **class-level** variables shared across all instances of the class, meaning there is only **one copy** of the variable for the entire class, accessible by all instances, which ensures consistency across them.
- They can be accessed directly using the class name.
- Characteristics:
 - Shared among all objects of the class.
 - Retain their value across method calls.
 - Useful for defining constants and counters.
- Code:

STATIC VARIABLES



STATIC METHODS

- Static methods can be called **without creating an instance** of the class, allowing direct access via the class name.
- These methods can only access other static members of the class and cannot interact with instance-specific data or methods.
- Characteristics:
 - Can access static variables and other static methods.
 - Cannot directly access instance variables or methods.
 - Cannot use the **this** keyword.
- Code:

STATIC METHODS

```
class StaticMethodExample
```

```
{
```

```
    int a = 0;
```

```
    static int b = 0;
```

```
    void showData()
```

```
    {
```

```
        System.out.println("Value of a : " + a);
```

```
        System.out.println("Value of b : " + b);
```

```
    }
```

```
    static void displayData()
```

```
    {
```

```
        StaticMethodExample obj = new StaticMethodExample();
```

```
        System.out.println("Value of a : " + obj.a);
```

```
        System.out.println("Value of b : " + b);
```

```
    }
```

```
    public static void main(String arg[])
```

```
    {
```

```
        displayData();
```

```
        StaticMethodExample obj = new StaticMethodExample();
```

```
        obj.showData();
```

```
    }
```

```
}
```

**A non-static method
can call static and
non-static variable
directly within class**

**A static
method
can call
static variable
directly but
non-static
variable by
use of object**

**A static method
can directly call
another static method
but non-static method
by use of object**

STATIC BLOCKS

- Static blocks are used to **initialize static variables** or perform any one-time initialization tasks when the class is loaded into memory.
- Static blocks are **executed only once**, when the class is first loaded, and they are executed in the order they appear in the source code.
- Characteristics:
 - A block that runs once when the class loads.
 - Used for complex initialization of static variables.
 - Executes before the **main()** method.
- Code:

STATIC NESTED CLASSES

- A nested class that does not need an outer object.
- Unlike non-static nested classes (inner classes), static nested classes do not have access to instance variables and methods of the outer class.
- They are conventionally used to logically group related classes or provide encapsulation.
- **Characteristics:**
 - Cannot access non-static members of the outer class directly.
 - Can only access outer class's static members directly.
 - Can be instantiated without an object of the outer class.
 - Useful for grouping classes that are only relevant to their outer class.
- **Code:**

STATIC NESTED CLASSES

```
public class Employee {
    String name;
    double salary;

    public double get_pay_for_period(double added_ot) {
        return salary + salary * added_ot;
    }

    static class OvertimeCalculator{
        private double added_percentage;
        public void set_rate (double rate) {
            added_percentage = rate;
        }

        public double get_rate() {
            return added_percentage;
        }
        public double get_overtime(double salary) {
            return salary * added_percentage;
        }
    }
}
```


ADVANTAGES OF USING STATIC MEMBERS

- **Memory Efficiency:** Shared members save memory as they don't require instance-specific storage.
- **Utility Methods:** Commonly used in helper classes like Math (e.g., `Math.sqrt()`).
- **Class Constants:** Define constants using `static final` for global access.

LIMITATIONS OF STATIC MEMBERS

- Cannot access instance-specific data directly.
- Static methods **cannot be overridden** (but can be hidden).
- Overusing static members can lead to tightly coupled code.

STATIC METHOD OVERLOADING

- Static methods can be **overloaded** just like instance methods (same name, different parameter lists).
- Code:

STATIC METHOD HIDING

- Static methods are **not overridden**; instead, they are **hidden** when a subclass defines a static method with the same signature.
- Code:

To be continued...

```
If(anyQuestions)
{
    askNow();
}
else
{
    thankYou();
    submitAttendance();
    endClass();
}
```